DATA SCIENCE

Stop Watching ML Courses. These 7 Algorithms Are Enough

The 7 algorithms behind most real machine learning work on business data. For each: how it works, when to use it, its weakness, and the code.

INSIDE THIS PDF · 9 PAGES

- 1. Linear Regression
- 2. Logistic Regression
- 3. Decision Tree
- 4. Random Forest
- 5. Gradient Boosting
- 6. K-Means
- 7. K-Nearest Neighbours

@thedataguy16

1. Linear Regression

Predicts a number by fitting a straight-line relationship.

```
from sklearn.linear_model import
LinearRegression
model = LinearRegression().fit(X_train, y_train)
print(model.coef_)   # effect of each feature
```

How it works

Finds the weights for each feature that minimise squared error. Price = a × area + b × rooms + c.

Use it for

Sales forecasts, price estimates, any time you also need to explain each feature's effect.

Weakness

Assumes a linear relationship. Sensitive to outliers and correlated features.

@thedataguy16

2. Logistic Regression

Predicts the probability of yes or no.

```
from sklearn.linear_model import
LogisticRegression
model = LogisticRegression(max_iter=1000)
model.fit(X_train, y_train)
proba = model.predict_proba(X_test)[:, 1]
```

How it works

A linear score passed through a sigmoid curve, giving a probability between 0 and 1.

Use it for

Churn, loan default, click or no click. The default first model for any yes/no problem.

Weakness

Misses complex interactions unless you add them as features. Scale features first.

@thedataguy16

3. Decision Tree

A set of if-then rules learned from data.

```
from sklearn.tree import DecisionTreeClassifier
model = DecisionTreeClassifier(max_depth=4)
model.fit(X_train, y_train)
```

How it works

Splits the data on the feature that best separates the outcome, then repeats on each branch.

Use it for

When business needs rules they can read: "if tenure < 6 months and complaints > 2, high churn risk".

Weakness

A single deep tree overfits badly. Limit max_depth.

@thedataguy16

4. Random Forest

Many decision trees voting together.

```
from sklearn.ensemble import
RandomForestClassifier
model = RandomForestClassifier(
    n_estimators=300, class_weight="balanced")
model.fit(X_train, y_train)
```

How it works

Trains hundreds of trees, each on a random sample of rows and features, then averages their votes.

Use it for

A strong default on tabular data with little tuning. Gives feature importance.

Weakness

Larger and slower than one tree. Harder to explain a single prediction.

@thedataguy16

5. Gradient Boosting

XGBoost, LightGBM, CatBoost. Usually the most accurate on tabular data.

```
from lightgbm import LGBMClassifier
model = LGBMClassifier(
    n_estimators=500, learning_rate=0.05)
model.fit(X_train, y_train)
```

How it works

Builds trees one after another. Each new tree corrects the errors of the ones before.

Use it for

When accuracy matters most: credit risk, demand forecasting, ranking.

Weakness

More settings to tune. Can overfit if learning rate is high and trees are many.

@thedataguy16

6. K-Means

Groups similar records when there is no label.

```
from sklearn.preprocessing import StandardScaler
from sklearn.cluster import KMeans
Xs = StandardScaler().fit_transform(X)
labels = KMeans(n_clusters=4,
n_init=10).fit_predict(Xs)
```

How it works

Places k centre points, assigns each record to the nearest, moves centres to the average, repeats.

Use it for

Customer segments by spend, frequency and recency. Then name each segment for the business.

Weakness

You choose k. Needs scaled features. Assumes round, similar-sized clusters.

@thedataguy16

7. K-Nearest Neighbours

Predicts from the most similar past records.

```
from sklearn.neighbors import
KNeighborsClassifier
model = KNeighborsClassifier(n_neighbors=7)
model.fit(Xs_train, y_train)
```

How it works

Finds the k closest records by distance and takes their majority vote or average.

Use it for

Simple baselines, "customers like you" style recommendations, small datasets.

Weakness

Slow on large data. Needs scaled features. Struggles with many features.

@thedataguy16

Which one to use

A quick decision guide for any new problem.

- **Predicting a number**
  Start with Linear Regression, then try Gradient Boosting.
- **Predicting yes or no**
  Start with Logistic Regression, then Random Forest, then Gradient Boosting.
- **Business needs readable rules**
  Decision Tree with limited depth.
- **No label, want groups**
  K-Means.
- **Small data, quick baseline**
  KNN.

Always: start simple, record the score, and only move to a complex model if it clearly beats the simple one.

@thedataguy16